



UNIVERSIDAD DON BOSCO

FACULTAD DE INGENIERÍA

DESARROLLO DE APLIC. WEB CON

SOFT. INTERPRET. EN EL SERVIDOR

(DSS404)

CICLO IV 2025

TÍTULO

Investigación Aplicada 1: Aplicaciones Server Side

DOCENTE:

Ing. Kevin Jiménez

NOMBRE	CARNÉ
Meléndez Áviles, Yassir Mauricio	MA243080
Hernández Guerrero, Guillermo Antonio	HG243075
Hernández Acevedo, Anderson Daniel	HA240610
Peña Bustillo, Bryan Alexis	PB243032
Alberto Guido, Christopher Adiel	AG252960

AULA VIRTUAL, 23 DE FEBRERO DE 2026

Contenido

1	¿Qué es la programación del lado del servidor (Server-Side)?	3
1.1	Definición y concepto fundamental.....	3
1.2	Diferencia entre Client-Side y Server-Side	3
1.3	¿Dónde y cómo se ejecuta el código Server-Side?	4
1.4	Rol del servidor en aplicaciones web	4
2	Cuadro comparativo de las principales características de lenguajes Server-Side	5
3	Bibliografía.....	8

1 ¿Qué es la programación del lado del servidor (Server-Side)?

1.1 Definición y concepto fundamental

“La programación en el lado del servidor se refiere al proceso de ejecutar scripts en un servidor web para generar contenido dinámico. Estos scripts interactúan con bases de datos y otros recursos para producir código en hypertext markup language (HTML), hojas de estilo en cascada (CSS) y JavaScript que pueden visualizarse en el navegador de un usuario.” (Lenovo, 2025).

Todo lo que suceda dentro del servidor es llamado procesamiento del lado del servidor, o server-side processing. Cuando tu aplicación necesita interactuar con el servidor (por ejemplo, para cargar o guardar datos), ésta realiza una petición del lado del cliente (client-side request) desde el navegador, a través de la red usando invocaciones remotas a métodos (remote procedure call, RPC). Mientras se está procesando una llamada RPC, tu servidor está ejecutando código del lado del servidor.

1.2 Diferencia entre Client-Side y Server-Side

La diferencia entre **Client-Side** y **Server-Side** se entiende mejor si pensamos en dónde ocurre cada proceso dentro de una aplicación web.

El **Client-Side** se refiere a todo lo que sucede en el navegador del usuario. Aquí se manejan la presentación y la interacción directa: el diseño de la página, los estilos, las animaciones y las validaciones rápidas de formularios. Lenguajes como HTML, CSS y JavaScript son los protagonistas. La ventaja es que la experiencia es más dinámica y rápida, pero la desventaja es que el código es visible y menos seguro.

El **Server-Side**, en cambio, ocurre en el servidor. Aquí se ejecuta la lógica de negocio, se procesan datos, se realizan consultas a la base de datos y se gestionan aspectos críticos como la autenticación y la seguridad. Lenguajes como Java, PHP, Python o Node.js suelen usarse en este ámbito. La ventaja es que el control y la seguridad son mayores, aunque puede ser más lento si el servidor está sobrecargado.

En resumen:

- **Client-Side** = lo que ves y con lo que interactúas directamente en tu navegador.

- **Server-Side** = lo que no ves, pero que hace posible que la aplicación funcione correctamente y de forma segura.

1.3 ¿Dónde y cómo se ejecuta el código Server-Side?

El código server-side se ejecuta en un servidor web o de aplicaciones, que puede ser físico (on-premise) o virtual/en la nube (AWS, Google Cloud, Azure, DigitalOcean, etc.). El flujo de ejecución sigue estos pasos fundamentales:

1	El usuario ingresa una URL o hace clic en un enlace → el navegador genera una Request HTTP.
2	La Request viaja por internet hasta el servidor donde está alojada la aplicación.
3	El servidor web (Apache, Nginx, etc.) recibe la petición y la pasa al intérprete o runtime del lenguaje.
4	El código server-side (PHP, Node.js, Python, etc.) se ejecuta: accede a BD, procesa lógica, genera respuesta.
5	El servidor construye una Response HTTP con el resultado (HTML, JSON, etc.) y la envía de vuelta.
6	El navegador del usuario recibe la respuesta y la renderiza para mostrarla al usuario.

1.4 Rol del servidor en aplicaciones web

El servidor (Server Side) en una aplicación web cumple el rol principal de procesar la información y gestionar toda la lógica interna del sistema. Básicamente es el encargado de recibir todas las solicitudes que envíe el usuario desde su navegador, generarlas y sacar una respuesta adecuada.

Otro rol importante del servidor es garantizar la seguridad de la aplicación web, protegiendo los datos de los usuarios mediante autenticación, autorización y cifrado de la información. Asimismo, controla el flujo de comunicación entre el cliente y el sistema, enviando respuestas en forma de páginas web o datos (como JSON) para que el navegador los muestre al usuario.

2 Cuadro comparativo de las principales características de lenguajes Server-Side

Características	PHP	Node.js (JavaScript del servidor)	Python (Django, Flask)	Java (JSP, Servlets)	Ruby (Ruby on Rails)	ASP.NET (C#)	.NET Core
Tipo de lenguaje	Interpretado (preprocesador de hipertexto)	Interpretado (Motor V8, Chrome/Node.js)	Lenguaje interpretado (Python) + frameworks web (Django, Flask)	Lenguaje compilado (Java) + tecnologías web (JSP, Servlets)	Lenguaje interpretado (Ruby) + framework web Rails	Lenguaje compilado (C#) + framework web ASP.NET	Lenguaje compilado (C#) + framework moderno .NET Core
Paradigma	Imperativo, OOP, Procedural, débilmente tipado	Event-driven (basado en eventos), Funcional, Asíncrono, callbacks/promesas	Multiparadigma: orientado a objetos, dinámico y flexible	Orientado a objetos, fuertemente tipado	Orientado a objetos, dinámico y flexible	Orientado a objetos, tipado fuerte	Orientado a objetos, tipado fuerte
Velocidad de ejecución	Media (depende del servidor PHP: Apache, Nginx)	Muy Alta (Motor V8 compilado JIT, event loop no bloqueante)	Media (interpretado, depende del servidor y optimización)	Alta (compilado, optimizado en JVM)	Media (interpretado, depende del servidor)	Alta (compilado y optimizado en .NET)	Muy alta (compilado, optimizado y multiplataforma)
Sintaxis	Clara, fácil de aprender, tags <?php ?>, similar a C/Java	JavaScript moderno: ES6+, async/await, destructuring, arrow functions	Clara, legible y cercana al lenguaje natural	Estricta, detallada y más verbosa	Muy legible, cercana al lenguaje natural	Estructurada y estricta	Estructurada, moderna y tipada

Productividad	Muy alta: desarrollo web rápido, scaffolding con Laravel/Symfony, debugging simple	Alta en grandes proyectos: TypeScript opcional, npm packages abundantes, pero curva aprendizaje asincrónica	Muy alta: desarrollo rápido gracias a librerías y frameworks	Alta en proyectos grandes, pero requiere más configuración	Muy alta para desarrollo rápido (prototipos y startups)	Alta en proyectos empresariales grandes	Muy alta con herramientas modernas y CLIExcelente y preparada para microservicios
Escalabilidad	Escalado vertical principalmente, limitado en conexiones simultáneas, requiere load balancing	Escalado horizontal excelente: Single-threaded event loop, sin bloqueos, ideal microservicios	Buena, soporta aplicaciones grandes con frameworks robustos	Excelente para sistemas empresariales y de gran escala	Buena, aunque requiere optimización en apps muy grandes	Excelente para sistemas grandes y empresariales	Excelente y preparada para microservicios
Arquitectura común	MVC (Laravel, Symfony, Yii), LAMP/LEMP stack tradicional	Microservicios, REST APIs, Backend for Frontend (BFF), event-driven (Ctrl + eventos)	MVC (Django), microservicios y APIs (Flask)	MVC, arquitectura basada en Servlets y JSP	MVC (Convención sobre configuración)	MVC, Web API, Blazor	MVC, Web API, Blazor, Microservicios
Seguridad	Buena: Laravel/Symfony incluyen CSRF, XSS, SQL	Muy robusta: OWASP Top 10 mitigación en frameworks	Buena: frameworks incluyen protección contra	Muy robusta: seguridad empresarial integrada en	Buena, incluye protección contra ataques comunes	Muy robusta (autenticación, roles, Identity)	Muy robusta (Identity, autenticación moderna, OAuth, JWT)

	Injection prevention, prepared statements, hash bcrypt	(Express.js Helmet), JWT/OAuth, input validation, rate limiting	ataques comunes (CSRF, XSS, SQL Injection)	el ecosistema Java			
Plataformas	Multiplataforma: Linux (principal), Windows (IIS), macOS (Apache/Nginx), shared hosting	Multiplataforma: Linux (producción), macOS (desarrollo), Windows (limitado), contenedores Docker	Multiplataforma (Linux, Windows, macOS)	Multiplataforma (gracias a la JVM)	Multiplataforma (Linux principalmente, también macOS/Windows)	Multiplataforma con .NET (Windows, Linux, macOS)	Multiplataforma real (Windows, Linux, macOS, contenedores)
Usos comunes	Blogs (WordPress), CMS (Drupal), aplicaciones CRUD rápidas, sitios compartidos de hosting	APIs REST modernas, Single Page Apps (SPA) backend, real-time apps (WebSockets), IoT, backend de startups	Aplicaciones web rápidas, APIs, prototipos, startups	Sistemas empresariales, aplicaciones corporativas, soluciones a gran escala	Startups, aplicaciones web rápidas, MVPs	Sistemas empresariales, APIs, aplicaciones corporativas	APIs modernas, microservicios, cloud, apps empresariales

3 Bibliografía

Cloudflare. (s.f.). *What do client side and server side mean? | Client side vs. server side.*

Obtenido de cloudflare.com:

<https://www.cloudflare.com/learning/serverless/glossary/client-side-vs-server-side/>

Lenovo. (26 de junio de 2025). *¿Qué es la programación en el lado del servidor?* Obtenido

de lenovo.com: [https://www.lenovo.com/mx/es/glosario/programacion-del-](https://www.lenovo.com/mx/es/glosario/programacion-del-servidor/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=ARcRdnrzI4q4QFF6U7TGFditRco2xix-eMjQ39aVk6oTgysLG6BeN1Xa)

[servidor/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=ARcRdnrzI4q4QFF6U7TGFditRco2xix-eMjQ39aVk6oTgysLG6BeN1Xa](https://www.lenovo.com/mx/es/glosario/programacion-del-servidor/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=ARcRdnrzI4q4QFF6U7TGFditRco2xix-eMjQ39aVk6oTgysLG6BeN1Xa)